Declarative Agentic Accountant System Design Specification

This document defines the architecture, data flows, and mechanics of the Declarative Agentic Accountant. The system uses a declarative engine to keep the code layer immutable. It abstracts browser selectors, endpoints, and task orders into a static text file. This design ensures that self-healing loops modify only structured configuration parameters, avoiding script corruption and runtime caching issues.

1. High Level Topology

The system is built around six primary components.

1. Orchestrator: This is the main pipeline coordinator. It manages the sequential execution loop, error interception, and self-healing tasks.
2. Step Interpreter: This is the execution engine. It parses JSON steps and executes corresponding browser automation, database queries, and email calls.
3. Declarative Profile: This is the modules_config JSON file. It contains the exact sequences, target URLs, search strings, and element selectors for every module.
4. Shared State Store: This is the shared_state SQLite database. It serves as the relational data layer for scraped transactions, active mapping rules, user context inputs, and system logs.
5. Persistent Browser Cache: This is the user session data directory. It stores persistent browser states, authentication cookies, and security certificates.
6. Web Control Panel: This is the FastAPI dashboard. It displays the reconciliation queue and allows users to input transaction contexts or approve proposed rules.

2. Shared State Database Design

The database maintains transaction lifecycles and historical context. It uses seven relational tables.

The raw transactions table stores scraped transaction records. It holds unique transaction identifiers, execution dates, payee strings, dollar amounts, categories, and account names.

The family context table stores scraped notes and email subject lines. It uses a key value structure where keys represent source identifiers and values hold plans, activities, or receipt items.

The benefits status table tracks health account status. It records employer health benefit categories and their corresponding dollar balances.

The review queue table serves as the state machine ledger. It tracks unmapped transactions, user context strings, LLM proposed mapping patterns, and the active progress state of the reconciliation loop.

The permanent mapping rules table stores deterministic routing patterns. It holds short keyword strings, target categories, and bill name associations for pre-approved matching logic.

The daily reports table stores the analytical briefings. It keeps a historical log of the plain text advisories, cash flow summaries, and optimization tips generated by the agent.

The audit log table tracks runtime metrics. It records module execution status, execution timestamps, and self-healing intervention logs.

3. The Declarative Engine and Execution Flow

Traditional scrapers rely on procedural scripts where selectors and sequence flows are hardcoded. A change in the target website layout breaks the script and requires manual code adjustments.
This system uses a declarative design. The step interpreter contains a predefined set of general actions. These actions include navigate, wait, click, fill, scrape ledger, scrape notes, scrape balances, and execute custom logic.
The pipeline execution follows a structured path.

1. The orchestrator reads the execution order from modules_config.
2. It launches a single Chromium instance using the persistent session folder on the host disk.
3. It passes the active browser page, the shared database connection, and the specific step list to the step interpreter.
4. The interpreter iterates through the step list, matching each action key to its corresponding internal execution block.
5. If all steps complete without errors, the orchestrator commits the status to the audit table and starts the next module.

This design decouples website layouts from the program logic. The Python interpreter remains completely static. Layout modifications require changes only to the string selectors in the JSON configuration file.

4. Self Healing Reflection Loop Mechanics

When a target website modifies its DOM layout, the step interpreter experiences an exception, such as a selector timeout. The self-healing loop recovers from these errors automatically.

1. The step interpreter throws an exception during execution.
2. The orchestrator intercepts the crash, halts the active step, and captures the complete traceback log.
3. The orchestrator reads the modules_config JSON file from disk.
4. It sends a repair prompt to Gemini containing the failed module name, the traceback log, and the entire current JSON configuration text.
5. The model analyzes the traceback, locates the broken step inside the JSON tree, identifies the failed selector, and updates the parameters.
6. The model returns only the repaired JSON configuration text.
7. The orchestrator performs a JSON validation check on the returned string. If the check passes, it writes the configuration back to disk.
8. The orchestrator re-reads modules_config, creates a fresh step interpreter instance, and attempts to execute the failed module a second time.

By updating only the JSON profile, the agent avoids Python syntax crashes, imports issues, and memory caching conflicts during hot-reloads.

5. Persistent Browser Context and Authentication

Automated login scripts often trigger security blocks and MFA requests. This system bypasses these issues by separating credential entry from automated runs.
The persistent browser context acts like a standard desktop browser. It writes session cookies, security tokens, and browser cache data directly to the host disk.
Initial authentication is managed through a manual process. The user opens the virtual desktop window using a web browser on Port 6080. The user manually logs into Quicken Simplifi, Google Keep, Forma, and Amazon, resolving any MFA or device verification prompts.
The remote services write authorization tokens and cookies into the browser context. Playwright automatically serializes and saves this security data.
During automated daily runs at 3:00 AM, the orchestrator launches Chromium using this pre-populated session folder. The browser transmits the saved cookies alongside the requests.
The remote platforms identify the session as an active, verified device. The sites bypass the login screens and load the data views immediately.
If a session expires, the scraper fails to find the data selectors. This triggers a timeout exception, which notifies the user to open Port 6080 and refresh the login state manually.

6. The Human in the Loop State Machine

When the reconciliation engine processes raw ledger rows, it uses a database-backed state machine to handle unmapped transactions.
The pending input state is triggered when a transaction has no matches in the permanent rules database. The engine writes the transaction to the review queue and alerts the user.
The input provided state is reached when the user reviews the item on the Port 8080 panel and inputs context regarding the purchase.
The rule proposed state occurs when the engine processes the user context alongside family notes and benefit balances. Gemini drafts a structured mapping pattern containing a text keyword, a category, and a target bill name.
The approved state is reached when the user verifies the rule on the dashboard. The web interface moves the rule into the permanent rules table and marks the item approved.
The approved rule protects the system from repeated API calls. On subsequent cycles, identical payees are resolved locally by the database without querying Gemini.

7. Interactive Write Back and Linkage Mechanics

After a transaction is approved, the system updates the online accounts to keep them in sync with the database.
For transaction reconciliation, the step interpreter opens Quicken Simplifi and loads the transaction ledger. It queries the local database for approved transaction IDs. For each matching row on the page, the interpreter locates the reviewed checkmark element and performs a click action to mark it reviewed on the live portal.
For bill management, the interpreter navigates to the bills view in Quicken Simplifi and scrapes the upcoming billers and amounts. It queries the database for unlinked raw transactions with matching dollar values.
When a match is found, the interpreter clicks the options trigger on the bill line, selects the linking option in the popup menu, types the transaction ID into the search input, checks the matching target row, and clicks confirm. This links the bank debit to the bill tracker, closing out the invoice cycle.